Reproducible Data Analysis

STAT 184 Assignment

Isabella Araujo De Moura

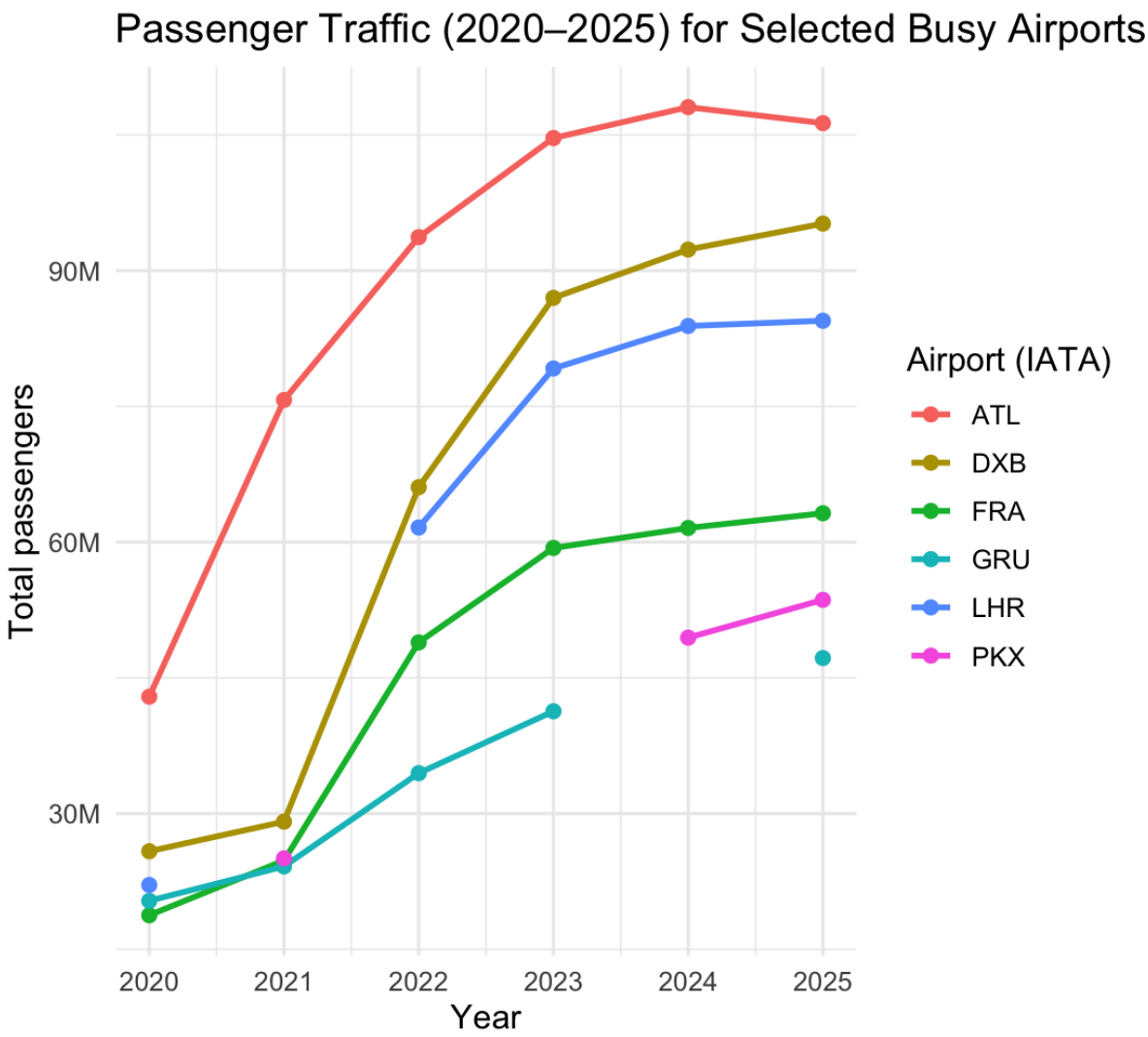
Invalid Date

Busiest Airport Analysis

For my analysis for comparing different airports and how busy they are, I had three different goals.
1. See how different airports have recovered from COVID-19, and if their numbers have stabilized.
2. See which airports are growing in numbers, which ones are already established and steady, and which are decreasing.
3. Compare geographically how different areas in the world differentiate in traffic.

For the airports 6 airports I chose to analyze:
1. Hartsfield-Jackson Atlanta International Airport
2. Frankfurt Airport
3. Beijing Daxing International Airport
4. São Paulo/Guarulhos International Airport
5. Dubai International Airport
6. Heathrow Airport

For my code, I did things pretty manually because I couldn't really figure out how to import data from the website. I essentially just tidied the data with all the names for the airport, and their passenger values for all the years. Then, I created a table with this tidied data, and created graphs for all of it.
I ended up with this graph:

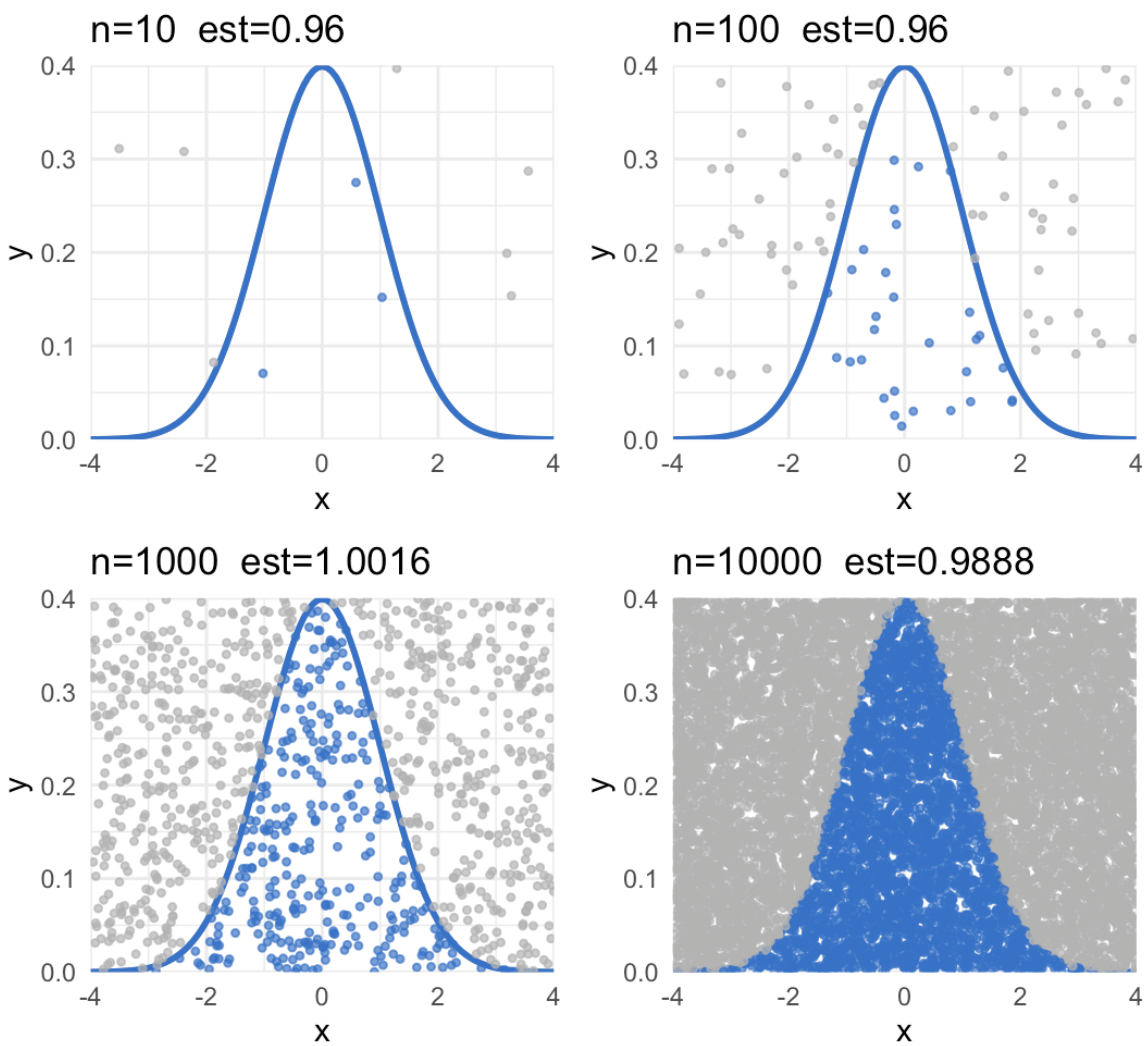


The conclusions I reached from this graph, were that the table shows that some of the airports don't have data for all the years. The bigger airports do but because each year the amount of visitors for each country fluctuates, some might not be on the list every year. The graph shows that the pandemic heavily affected airport numbers, and in they rose a lot from 2021 until 2024 where it stabilized. What I've noticed is that the airports with the most visitors, are usually central hubs to regions. Dubai airport is big because its in the middle east and you can access almost anywhere from it. Heathrow is also huge because its in Europe and you can access a lot of places. The biggest is Atlanta airport because its in the southern US and you can go anyone in the Americas from it. The thing that determines if in airport is big the most is if they're in a good spot to be a hub.

Monte Carlo Numerical Integration

In this problem, the goal was to create an algorithm that can approximate an integral for a function using the monte carlo process, by using a lot of randomly placed dots to estimate that value.
My goals were:
1. First, create a function that can randomly generate points with specified bounds to fit within the function, and compute the area based on that.
2. Develop a way to identify where the points land based on the specified function.
3. Make the function be able to estimate the area under the curve based on those points.

For my code, firstly I created a simulator that generates the points with specified max and min bounds. Then, I made a part that identifies if points are underneath the function or not, and estimates the function based on that. Afterwards, it creates several graphs based on the estimates for the value, and how many points are used in each. The resulting graph is this:



My analysis of this graph was that surprisingly math can be estimated. While we can use strict algorithms and formulas to find the values and accurate information from integrating, there's a lot of methods to find things by estimation. It does take a lot of simulations to eventually get close to the right answer because there's quite a lot of inaccuracy at first, but it gets there. Based on the estimates from the graph, the value should be 1. I do think someone else would arrive at the same value considering how close to 1 it got.

Planning and Prompting GenAI Tools

In this part, we were prompted to use AI to tidy and analyze the calcium.csv file, which analyses the calcium levels in women's arms, depending if they received treatment to improve

their calcium levels over 3 years.
My plan to tidy, is to firstly reorganized the table to have things be properly labeled and organized, and make sure that every data point follows tidy principles. Some columns have repeated names so values aren't unique, and it's not organized in a way that makes data analysis easy. Then, my analysis would be to create a line graph that shows the difference for the average for each group over time, and how the treatment may have affected the women.

AI Response

1) What's "untidy" about calcium.csv (and what it should look like) Right now the file is wide and effectively contains two separate tables glued side-by-side:

Columns 1–4: the null treatment group measured at y0, y1, y2, y3 Columns 5–8: the diet/exercise group measured at y0, y1, y2, y3 The header repeats (y0,y1,y2,y3,y0,y1,y2,y3), so the columns are not uniquely named. Each row contains measurements from two different women (one in each group), so "one observational unit per row" is violated. The last row has missing values in the first group and values in the second group, which also signals the two groups have different counts (15 vs 16). A tidy structure should have:

One row = one woman at one time point Separate columns for: subject_id (unique woman identifier) group (null vs treatment) year (0,1,2,3) calcium (the measured value) Example of the target tidy format:

subject_id group year calcium N01 null 0 87.3 N01 null 1 86.9 … … … … T01 treatment 0 83.8 T01 treatment 1 85.5 2) Tidy plan (step-by-step, using AI or code) Step A — Import without changing the raw file

Read the CSV exactly as-is (including the repeated headers). Step B — Rename columns so they're unique

Rename columns 1–4 to null_y0, null_y1, null_y2, null_y3 Rename columns 5–8 to treat_y0, treat_y1, treat_y2, treat_y3 Step C — Split into two person-level tables

null_df: keep the null_ columns, assign group = "null", create subject ids N01..N15 treat_df: keep the treat_ columns, assign group = "treatment", create subject ids T01..T16 Drop rows that are completely missing for a given group (e.g., the null part of the last line). Step D — Pivot longer (wide → long)

Convert *_y0..*_y3 into: year (0–3) calcium (numeric) Remove the prefix in the pivot step so year comes out clean. Step E — Combine the two long tables

Row-bind null and treatment long data into one tidy dataset. Step F — Sanity checks

Counts you should expect if complete: null group: 15 women × 4 years = 60 rows treatment group: 16 women × 4 years = 64 rows total: 124 rows Check missingness by group and year. 3)

Analysis: what to compute and why Because this is a longitudinal (repeated measures) dataset, the most direct "exploratory" analysis aligned with your plan is:

Group-by-time summary For each group and year, compute: mean_calcium sd_calcium n (non-missing count) optionally se = sd/sqrt(n) for error bars Plot: mean calcium over time by group X-axis: year (0,1,2,3) Y-axis: mean ulnar calcium Color/line type: group (null vs treatment) Add points at each year Add uncertainty (e.g., ±1 SE or a 95% CI ribbon/error bars) This plot directly answers: Do the average trajectories differ? (levels and slopes)

(Optional but strong) Change-from-baseline For each woman, compute: change = calcium_year_k - calcium_year_0 Then summarize mean change by group and year. This helps because baseline values can differ between groups; change scores focus on within-person trends. (Optional inference) Repeated measures modeling If your assignment allows inferential stats: Fit a model like calcium ~ group * year + (1 | subject_id) (mixed effects), or a repeated-measures ANOVA equivalent. The key term is group:year (whether the trend over time differs by group). 4) What the provided raw values suggest (interpretation consistent with the data layout) From the numbers you shared, a consistent pattern appears:

Null group values often decline over time (many y3 values are lower than y0). Diet/exercise (treatment) group also declines in many cases, but often appears higher at later years than the null group (e.g., y2/y3 are frequently less depressed). So, the most likely story your mean-lines plot will reveal is:

Both groups trend downward across 3 years (bone mineral/calcium decreasing with time/aging), but the treatment group's decline may be smaller (a "less steep" downward slope), or it may stabilize better. That "difference in slopes" is exactly what your line-graph plan is designed to show.

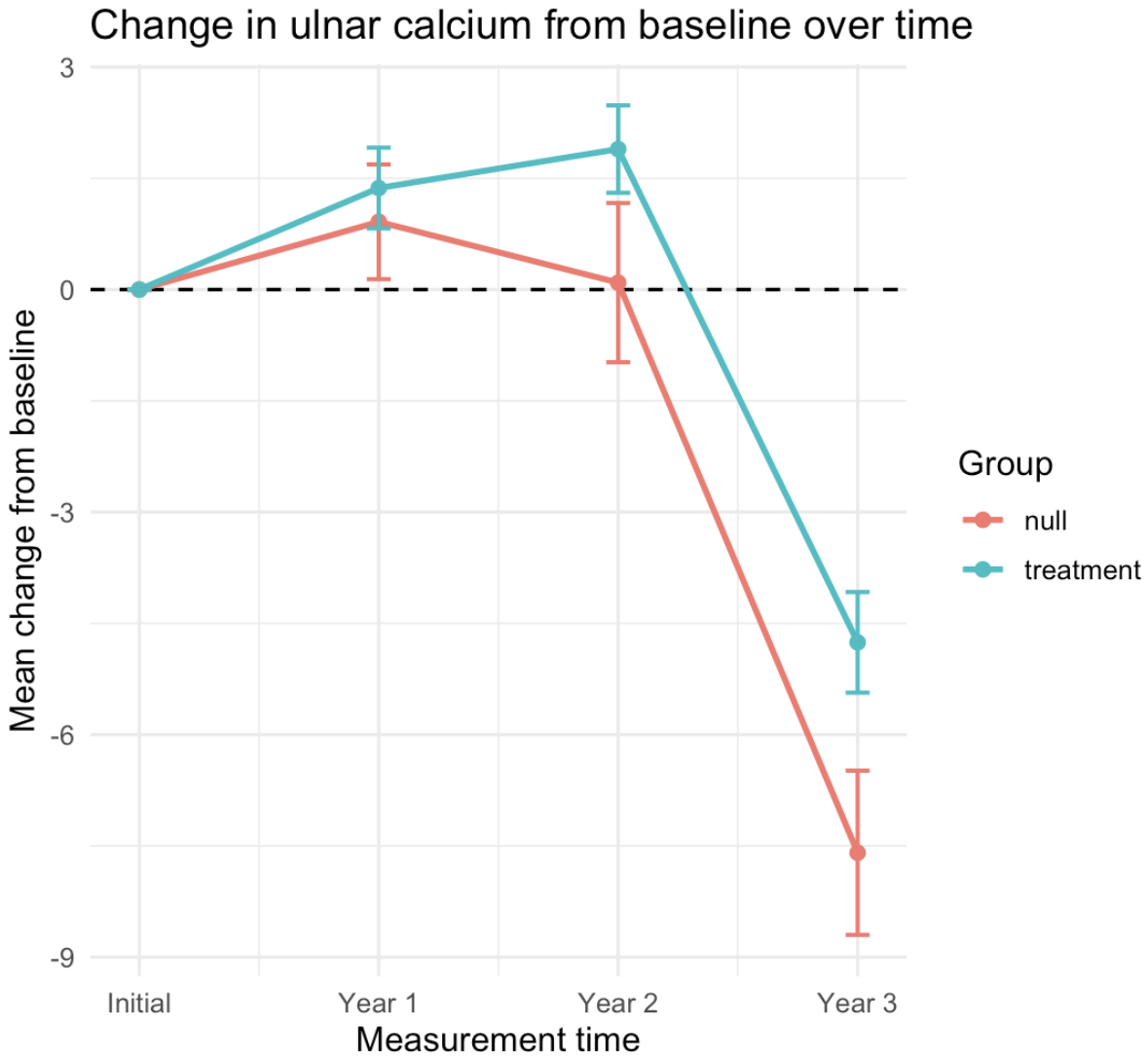
5) The deliverable plot(s) to include Minimum (matches your plan):

Line plot of group means vs year Two lines (null, treatment) Points at each year Error bars (SE or CI) Recommended add-on (often makes conclusions clearer): 2) Line plot of mean change-from-baseline vs year

Starts at 0 for both groups at year 0 Shows divergence cleanly 6) Notes about data quality and reporting The last CSV row has missing entries for the null group but has treatment values. That is expected given the stated group sizes (15 vs 16). When tidying, you'll handle this by dropping null-missing rows only for the null table. In your write-up, explicitly state: the data were reshaped to long format, the unit of observation became "woman-year," and summaries used non-missing values by group/year. If you tell me what tool you're expected to use (R/tidyverse, Python/pandas, Excel, etc.), I can generate the exact tidying + plotting code in that language to match this plan.

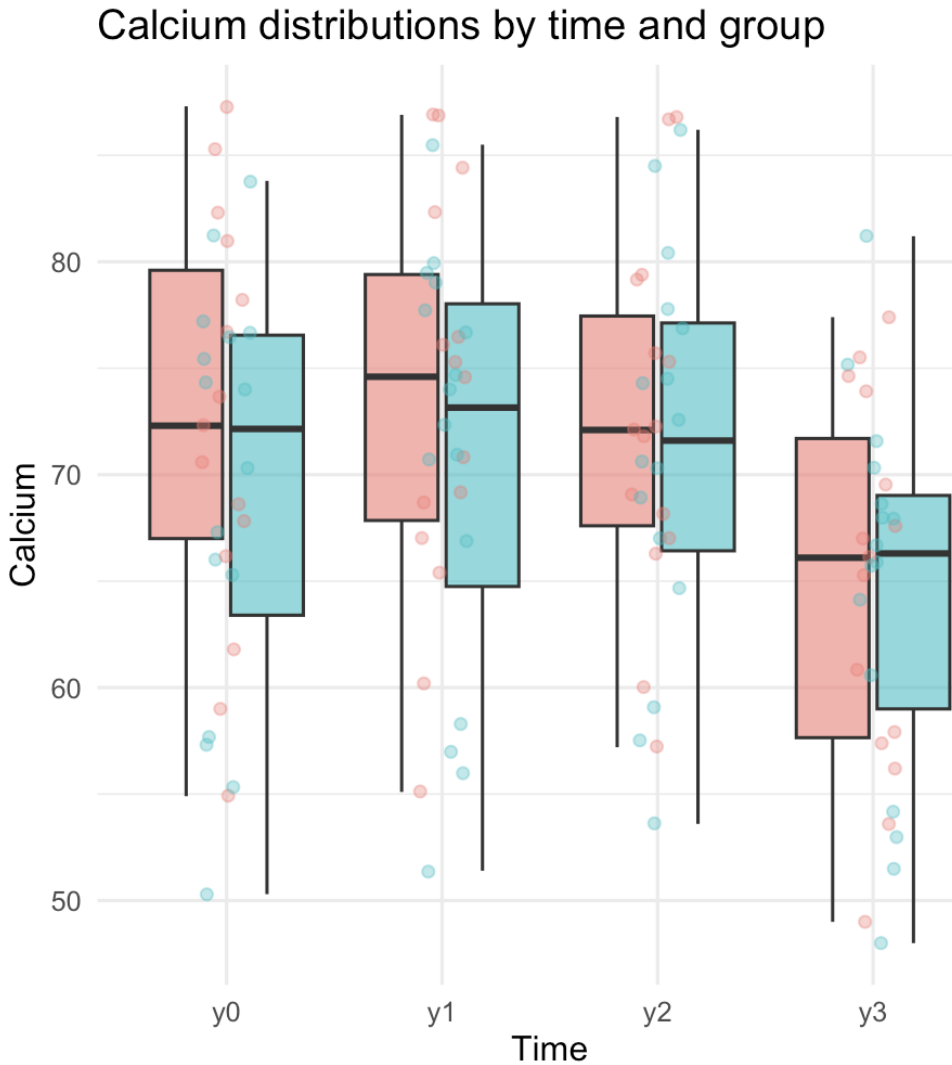
Second Prompt

Afterwards, I asked it for R code to properly conduct this analysis. The final resulting graph was this:



Generic Response

When just asking the AI in a random prompt for purely visualizng the data without giving it any



context at all, this is the graph I got:

Now as can be seen, both graphs actually do a pretty good job at analyzing the data. It's pretty apparent that the AI makes a lot of assumptions when it doesn't know exactly what its analyzing. The titles in the one with context are a lot more fleshed out, and overall the labels are better done and it makes more sense, but I'm honestly surprised that the AI was able to infer quite a bit about what to do for the analysis. My idea was to make a line graph to make a simpler graph visually that can easily indicate patterns, but without the context, the AI decided to use a box and whisker plot that is also good in this situation to represent overall trends. I think that it's important to be critical of AI generated code because it's pretty easy for it to misrepresent and hallucinate things, and while it worked well here, there might be situations where it doesn't.

Reflection

I've genuinely learned quite a bit from this course. I've realized how complex data analysis is, and how many little subconscious things that we don't even realize when we're looking at a graph, that are intentional chooses when designing graphs. Tidying data has also become something that I've always been trying to pay attention to because of how important it's become when analyzing data in class, since if data isn't tidy, then it's really hard to actually convert it into graphs and something more feasible. Something really interesting I've also learned from the lectures is how small details can affect how people perceive things. People tend to subconsciously consume data in a specific order, and go from largest to smallest and if that's not taken to account, it can make graphs really confusing. I've also learned about a lot of types of graphs that I never heard of before, like geographic graphs and more, which can make for really interesting versions of data representation. I've really tried my best to implement these things into my new assignments, and it genuinely makes me proud of myself that I've been able to improve things so much and make it better. The past few assignments I've tried my best to implement these things, and I think my code has shown that I've tried to really try to implement tidy data.

AI

For the AI that I used for the AI question, I used ChatGPT 5.2 on April 12th 2026.
The prompts I used:
In this part, we were prompted to use AI to tidy and analyze the calcium.csv file, which analyses the calcium levels in women's arms, depending if they received treatment to improve their calcium levels over 3 years. My plan to tidy, is to firstly reorganized the table to have things be properly labeled and organized, and make sure that every data point follows tidy principles. Then, my analysis would be to create a line graph that shows the difference for the average for each group over time, and how the treament may have affected the women. Planning and Prompting Gen AI: Get the Data For this portion of the assignment, you will need to refer to the Calcium data found in this CSV file: calcium.csv. Note: clicking this link will automatically download the CSV file. These data come from a longetudinal study that followed 31 women over the course of four years. At the start of the study, researchers measured the amount of calcium in the ulna from each woman's dominant arm through photon absorptiometry. Throughout the next four years, 15 of the women went about their lives as they typically did; they make up the null treatment group haivng not received any special treatment. The other 16 women spent the next four years receiving special help with their diet and completing a regular exercise program; they make up the diet/exercise treatment group. Once a year each of thee 31 women had their ulnar calcium levels measured. In the calcium.csv file, you'll find the measurements (Initial, Year 1, Year 2, Year 3) for each woman; the null

treatment group take up the first four columns and the diet/exercise group make up the second four columns. From the last sentence you should be able to state that these data are not tidy. You SHOULD NOT alter the calcium.csv file in any way before uploading it to your chosen genAI/LLM tool. You need to make a full plan to not only tidy these data but to then create a plot/graph that would help you explore the difference in ulnar calcium measurements across time and the two treatment groups.

Given this plan and information, create an analysis of the data.
And then: use R/tidyverse to perform this.

The other prompt I used for the generic version was:
Create a data visualization for this data.

For the responses given from the AI: Copilot said:
1) What's "untidy" about `calcium.csv` (and 1) What's "untidy" about calcium.csv (and what it should look like) Right now the file is wide and effectively contains two separate tables glued side-by-side:

Columns 1–4: the null treatment group measured at y0, y1, y2, y3 Columns 5–8: the diet/exercise group measured at y0, y1, y2, y3 The header repeats (y0,y1,y2,y3,y0,y1,y2,y3), so the columns are not uniquely named. Each row contains measurements from two different women (one in each group), so "one observational unit per row" is violated. The last row has missing values in the first group and values in the second group, which also signals the two groups have different counts (15 vs 16). A tidy structure should have:

One row = one woman at one time point Separate columns for: subject_id (unique woman identifier) group (null vs treatment) year (0,1,2,3) calcium (the measured value) Example of the target tidy format:

subject_id group year calcium N01 null 0 87.3 N01 null 1 86.9 … … … … T01 treatment 0 83.8 T01 treatment 1 85.5 2) Tidy plan (step-by-step, using AI or code) Step A — Import without changing the raw file

Read the CSV exactly as-is (including the repeated headers). Step B — Rename columns so they're unique

Rename columns 1–4 to null_y0, null_y1, null_y2, null_y3 Rename columns 5–8 to treat_y0, treat_y1, treat_y2, treat_y3 Step C — Split into two person-level tables

null_df: keep the null_ columns, assign group = "null", create subject ids N01..N15 treat_df: keep the treat_ columns, assign group = "treatment", create subject ids T01..T16 Drop rows that are completely missing for a given group (e.g., the null part of the last line). Step D — Pivot longer (wide → long)

Convert *_y0..*_y3 into: year (0–3) calcium (numeric) Remove the prefix in the pivot step so year comes out clean. Step E — Combine the two long tables

Row-bind null and treatment long data into one tidy dataset. Step F — Sanity checks

Counts you should expect if complete: null group: 15 women × 4 years = 60 rows treatment group: 16 women × 4 years = 64 rows total: 124 rows Check missingness by group and year. 3) Analysis: what to compute and why Because this is a longitudinal (repeated measures) dataset, the most direct "exploratory" analysis aligned with your plan is:

Group-by-time summary For each group and year, compute: mean_calcium sd_calcium n (non-missing count) optionally se = sd/sqrt(n) for error bars Plot: mean calcium over time by group X-axis: year (0,1,2,3) Y-axis: mean ulnar calcium Color/line type: group (null vs treatment) Add points at each year Add uncertainty (e.g., ±1 SE or a 95% CI ribbon/error bars) This plot directly answers: Do the average trajectories differ? (levels and slopes)

(Optional but strong) Change-from-baseline For each woman, compute: change = calcium_year_k - calcium_year_0 Then summarize mean change by group and year. This helps because baseline values can differ between groups; change scores focus on within-person trends. (Optional inference) Repeated measures modeling If your assignment allows inferential stats: Fit a model like calcium ~ group * year + (1 | subject_id) (mixed effects), or a repeated-measures ANOVA equivalent. The key term is group:year (whether the trend over time differs by group). 4) What the provided raw values suggest (interpretation consistent with the data layout) From the numbers you shared, a consistent pattern appears:

Null group values often decline over time (many y3 values are lower than y0). Diet/exercise (treatment) group also declines in many cases, but often appears higher at later years than the null group (e.g., y2/y3 are frequently less depressed). So, the most likely story your mean-lines plot will reveal is:

Both groups trend downward across 3 years (bone mineral/calcium decreasing with time/aging), but the treatment group's decline may be smaller (a "less steep" downward slope), or it may stabilize better. That "difference in slopes" is exactly what your line-graph plan is designed to show.

5) The deliverable plot(s) to include Minimum (matches your plan):

Line plot of group means vs year Two lines (null, treatment) Points at each year Error bars (SE or CI) Recommended add-on (often makes conclusions clearer): 2) Line plot of mean change-from-baseline vs year

Starts at 0 for both groups at year 0 Shows divergence cleanly 6) Notes about data quality and reporting The last CSV row has missing entries for the null group but has treatment values. That is expected given the stated group sizes (15 vs 16). When tidying, you'll handle this by dropping null-missing rows only for the null table. In your write-up, explicitly state: the data were reshaped to long format, the unit of observation became "woman-year," and summaries used non-missing values by group/year. If you tell me what tool you're expected

to use (R/tidyverse, Python/pandas, Excel, etc.), I can generate the exact tidying + plotting code in that language to match this plan.

Below is a complete R / tidyverse workflow that (1) imports the CSV as-is, (2) tidies it into long format with one row per woman-year, (3) computes group-by-year summaries, and (4) makes the mean trajectory line plot (plus an optional change-from-baseline plot). library(tidyverse)

1) Read the raw file (do not edit the CSV beforehand)

```
library(readr)
raw <- readr::read_csv("calcium.csv", col_types = cols(.default = col_double()))
```

```
New names:
* `y0` -> `y0...1`
* `y1` -> `y1...2`
* `y2` -> `y2...3`
* `y3` -> `y3...4`
* `y0` -> `y0...5`
* `y1` -> `y1...6`
* `y2` -> `y2...7`
* `y3` -> `y3...8`
```

2) Give unique, informative column names

First 4 columns = null group; last 4 = diet/exercise group

```
colnames(raw) <- c("null_y0","null_y1","null_y2","null_y3", "treat_y0","treat_y1","treat_y2"
```

3) Split into two tables (one woman per row), assign subject IDs, and drop rows that are entirely missing for that group

```
library(dplyr)
```

```
Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```r
null_wide <- raw |> select(starts_with("null_")) |> filter(if_any(everything(), ~ !is.na(.x)
# keep rows with at least one non-NA
mutate(subject_id = sprintf("N%02d", row_number()), group = "null")
treat_wide <- raw |> select(starts_with("treat_")) |> filter(if_any(everything(), ~ !is.na(.x
```

4) Pivot longer into tidy form: one row = one woman at one year

```r
library(tidyr)
null_long <- null_wide |> pivot_longer( cols = starts_with("null_y"), names_to = "year", valu

treat_long <- treat_wide |> pivot_longer( cols = starts_with("treat_y"), names_to = "year", v

calcium_tidy <- bind_rows(null_long, treat_long) |> arrange(group, subject_id, year)
```

Optional: quick checks

```r
calcium_tidy |> count(group, year)
```

```
# A tibble: 8 x 3
  group      year     n
  <chr>     <int> <int>
1 null          0    15
2 null          1    15
3 null          2    15
4 null          3    15
5 treatment     0    16
6 treatment     1    16
7 treatment     2    16
8 treatment     3    16
```

```r
calcium_tidy |> count(group, subject_id)
```

```
# A tibble: 31 x 3
   group subject_id     n
   <chr> <chr>      <int>
 1 null  N01            4
 2 null  N02            4
 3 null  N03            4
 4 null  N04            4
 5 null  N05            4
```

```
 6 null  N06            4
 7 null  N07            4
 8 null  N08            4
 9 null  N09            4
10 null  N10            4
# i 21 more rows
```

5) Summarize: mean calcium by group and year (with SE for error bars)

```
summary_by_group_year <- calcium_tidy |> group_by(group, year) |> summarise( n = sum(!is.na(

summary_by_group_year
```

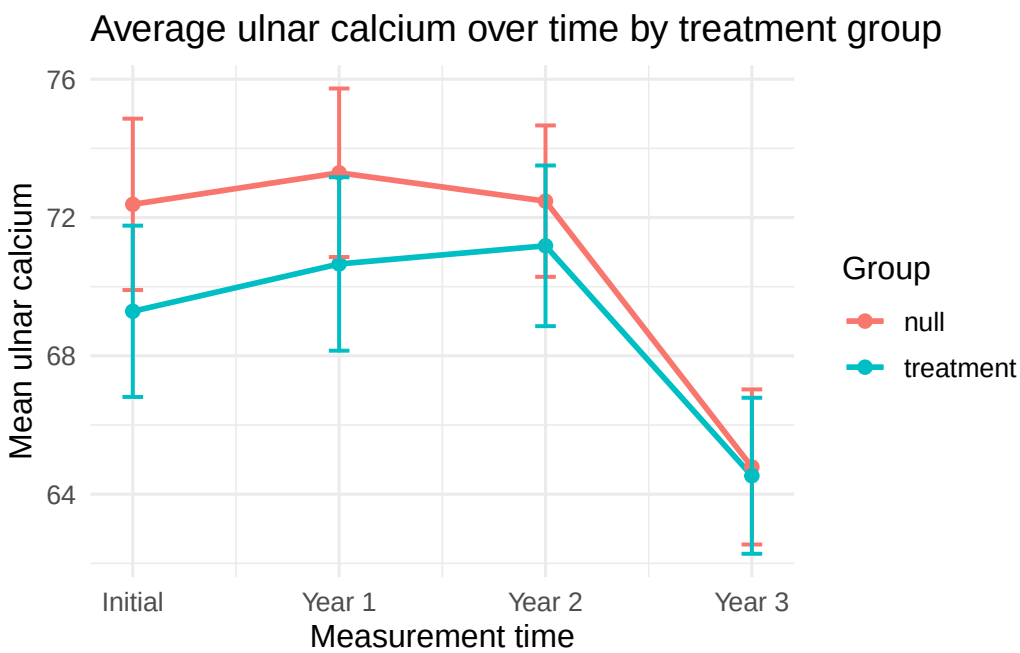
```
# A tibble: 8 x 6
  group      year     n  mean    sd    se
  <chr>     <int> <int> <dbl> <dbl> <dbl>
1 null          0    15  72.4  9.60  2.48
2 null          1    15  73.3  9.44  2.44
3 null          2    15  72.5  8.48  2.19
4 null          3    15  64.8  8.69  2.24
5 treatment     0    16  69.3  9.91  2.48
6 treatment     1    16  70.7 10.0   2.51
7 treatment     2    16  71.2  9.29  2.32
8 treatment     3    16  64.5  9.02  2.26
```

Mean-over-time plot (your primary analysis) library(tidyverse)

```
library(tidyverse)
```

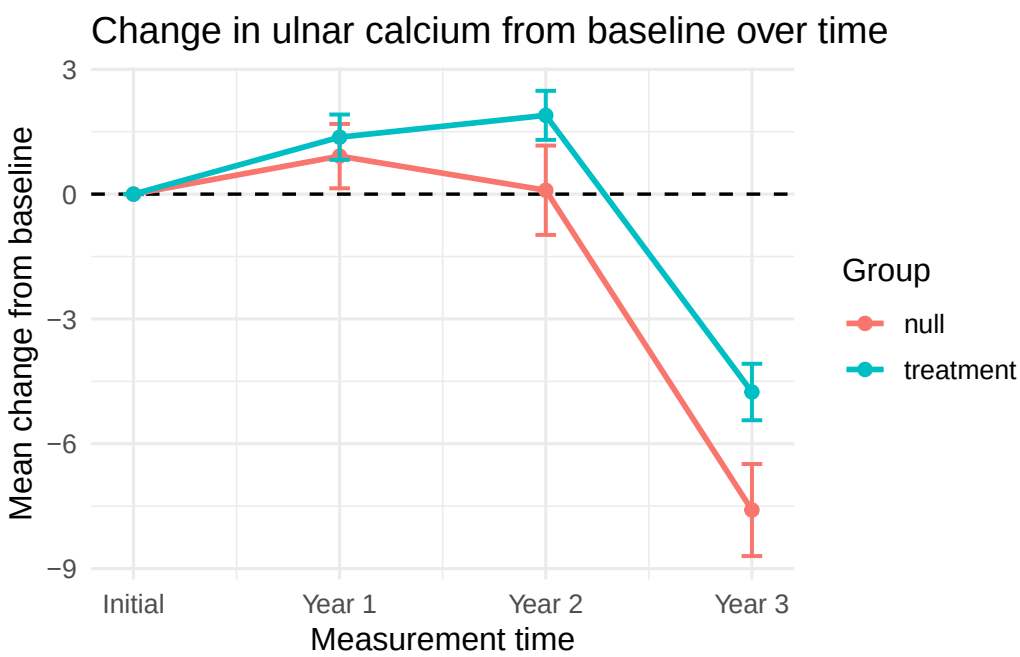
```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v forcats   1.0.1     v purrr     1.2.1
v ggplot2   4.0.2     v stringr   1.6.0
v lubridate 1.9.5     v tibble    3.3.1
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becom
```

```r
ggplot(summary_by_group_year, aes(x = year, y = mean, color = group, group = group)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  geom_errorbar(aes(ymin = mean - se, ymax = mean + se), width = 0.1, linewidth = 0.7) +
  scale_x_continuous(breaks = 0:3, labels = c("Initial", "Year 1", "Year 2", "Year 3")) +
  labs(
    title = "Average ulnar calcium over time by treatment group",
    x = "Measurement time",
    y = "Mean ulnar calcium",
    color = "Group"
  ) +
  theme_minimal(base_size = 12)
```




```r
library(tidyverse)

change_tidy <- calcium_tidy %>%
  group_by(subject_id, group) %>%
  mutate(baseline = calcium[year == 0][1],
         change = calcium - baseline) %>%
  ungroup()

change_summary <- change_tidy %>%
```

```r
  group_by(group, year) %>%
  summarise(
    n = sum(!is.na(change)),
    mean_change = mean(change, na.rm = TRUE),
    sd = sd(change, na.rm = TRUE),
    se = sd / sqrt(n),
    .groups = "drop"
  )

ggplot(change_summary, aes(x = year, y = mean_change, color = group, group = group)) +
  geom_hline(yintercept = 0, linetype = "dashed", linewidth = 0.6) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  geom_errorbar(aes(ymin = mean_change - se, ymax = mean_change + se), width = 0.1, linewidtl
  scale_x_continuous(breaks = 0:3, labels = c("Initial", "Year 1", "Year 2", "Year 3")) +
  labs(
    title = "Change in ulnar calcium from baseline over time",
    x = "Measurement time",
    y = "Mean change from baseline",
    color = "Group"
  ) +
  theme_minimal(base_size = 12)
```



What to write in your interpretation (based on what the plot will show) Describe the overall trend across time (often declining in longitudinal bone/calcium measures). Compare groups by: starting level (baseline means), slope/trajectory (does treatment decline less?), and whether uncertainty bars overlap (exploratory, not definitive). If you paste the output of summary_by_group_year (the printed tibble), I can help you write a tight paragraph interpreting the exact numeric differences your plot is showing.

```
library(readr)
library(dplyr)
library(tidyr)
library(ggplot2)

df <- read_csv("calcium.csv", show_col_types = FALSE)
```

```
New names:
* `y0` -> `y0...1`
* `y1` -> `y1...2`
* `y2` -> `y2...3`
* `y3` -> `y3...4`
* `y0` -> `y0...5`
* `y1` -> `y1...6`
* `y2` -> `y2...7`
* `y3` -> `y3...8`
```

Your CSV has duplicate headers; rename to unique names

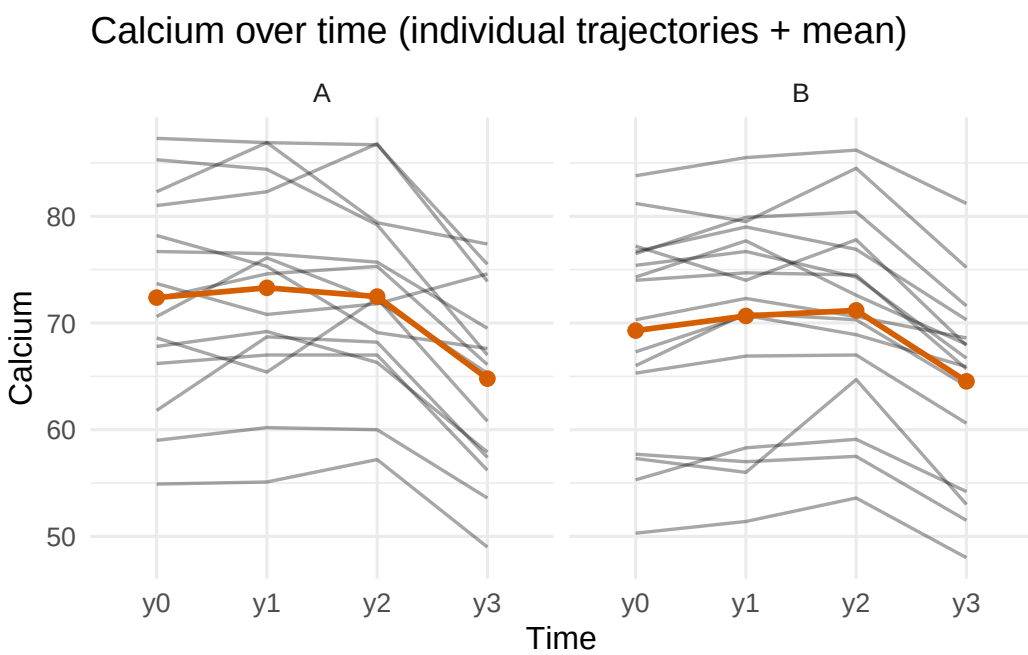
```
names(df) <- c("a_y0","a_y1","a_y2","a_y3","b_y0","b_y1","b_y2","b_y3")
```

Reshape to long format (handles blanks as NA)

```
long <- df |>
  mutate(subject = row_number()) |>
  pivot_longer(cols = -subject, names_to = "col", values_to = "value") |>
  mutate(
    group = if_else(startsWith(col, "a_"), "A", "B"),
    time  = sub("^[ab]_", "", col),
    time  = factor(time, levels = c("y0","y1","y2","y3"))
  ) |>
  select(subject, group, time, value)
```

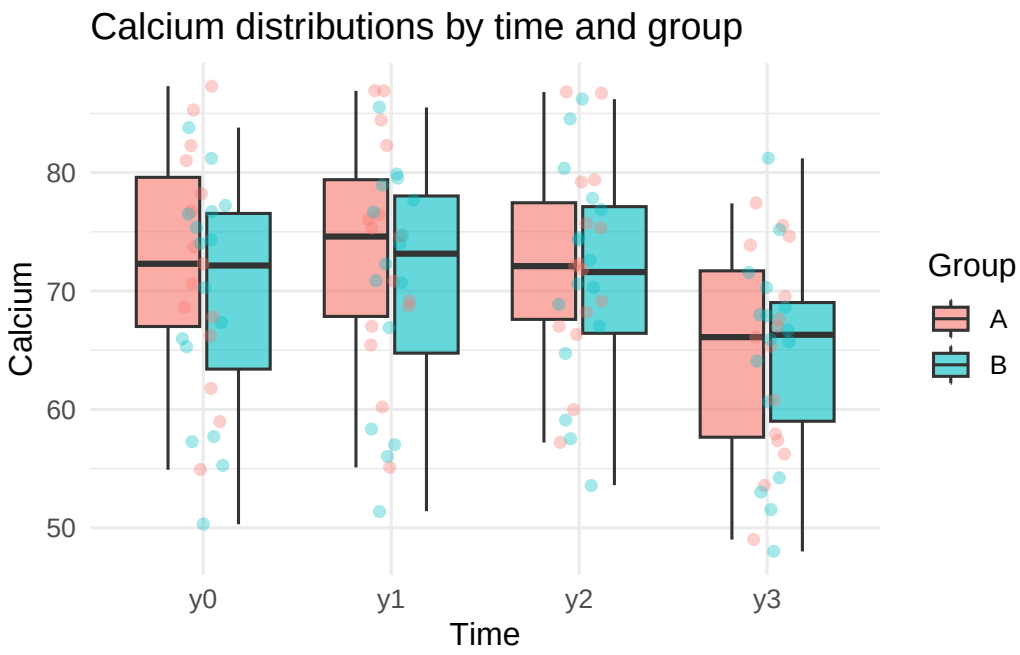
1) Small multiples: individual trajectories + group mean

```r
p1 <- ggplot(long, aes(x = time, y = value, group = subject)) +
  geom_line(alpha = 0.35, linewidth = 0.6, na.rm = TRUE) +
  stat_summary(
    aes(group = 1),
    fun = mean, geom = "line",
    linewidth = 1.0, colour = "#D55E00",
    na.rm = TRUE
  ) +
  stat_summary(
    aes(group = 1),
    fun = mean, geom = "point",
    size = 2.2, colour = "#D55E00",
    na.rm = TRUE
  ) +
  facet_wrap(~group) +
  labs(
    title = "Calcium over time (individual trajectories + mean)",
    x = "Time", y = "Calcium"
  ) +
  theme_minimal(base_size = 12)

print(p1)
```



2) Optional: distribution summary (boxplot + jitter)

```
p2 <- ggplot(long, aes(x = time, y = value, fill = group)) +
  geom_boxplot(outlier.shape = NA, alpha = 0.6, na.rm = TRUE) +
  geom_jitter(aes(color = group),
              width = 0.12, alpha = 0.35, size = 1.6,
              na.rm = TRUE, show.legend = FALSE) +
  labs(
    title = "Calcium distributions by time and group",
    x = "Time", y = "Calcium", fill = "Group"
  ) +
  theme_minimal(base_size = 12)

print(p2)
```



Code Appendix

Code for airports:

```
library(dplyr)
library(tidyr)
library(scales)
```

```
Attaching package: 'scales'

The following object is masked from 'package:purrr':

    discard

The following object is masked from 'package:readr':

    col_factor
```

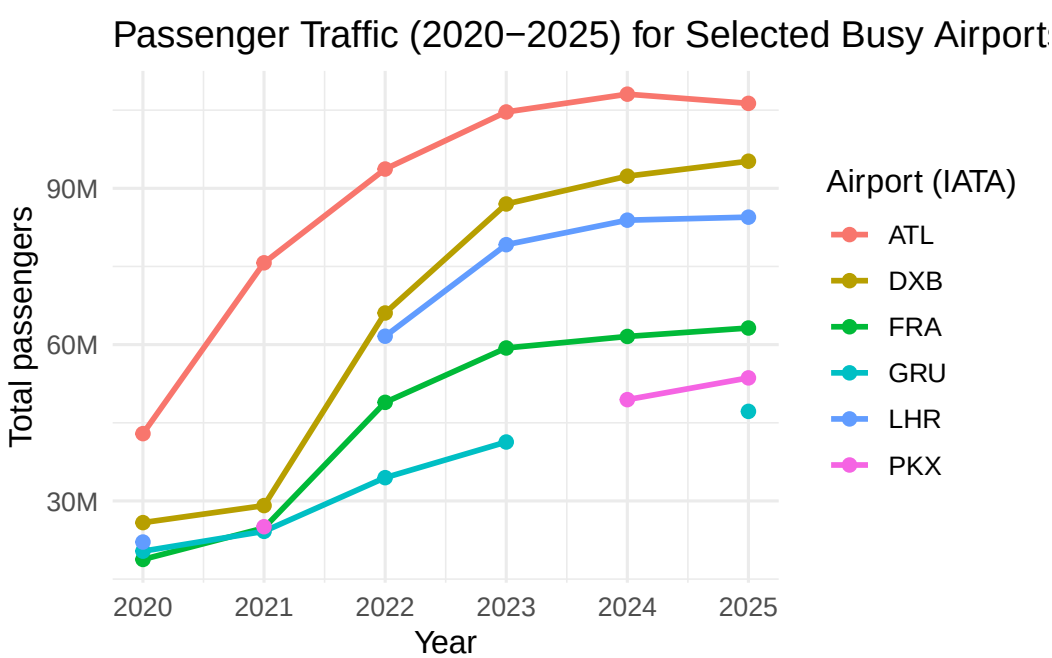
```r
# Create vector with all the years, airport codes, airport names, and the passenger numbers.
busiest_airports_tidy <- data.frame(
  year = c(
    2020,2020,2020,2020,2020,2020,
    2021,2021,2021,2021,2021,2021,
    2022,2022,2022,2022,2022,2022,
    2023,2023,2023,2023,2023,2023,
    2024,2024,2024,2024,2024,2024,
    2025,2025,2025,2025,2025,2025
  ),
  iata = c(
    "ATL","FRA","PKX","GRU","DXB","LHR",
    "ATL","FRA","PKX","GRU","DXB","LHR",
    "ATL","FRA","PKX","GRU","DXB","LHR",
    "ATL","FRA","PKX","GRU","DXB","LHR",
    "ATL","FRA","PKX","GRU","DXB","LHR",
    "ATL","FRA","PKX","GRU","DXB","LHR"
  ),
  airport = c(
    "Hartsfield-Jackson Atlanta International Airport",
    "Frankfurt Airport",
    "Beijing Daxing International Airport",
    "São Paulo/Guarulhos International Airport",
    "Dubai International Airport",
    "Heathrow Airport",

    "Hartsfield-Jackson Atlanta International Airport",
    "Frankfurt Airport",
    "Beijing Daxing International Airport",
    "São Paulo/Guarulhos International Airport",
    "Dubai International Airport",
    "Heathrow Airport",

    "Hartsfield-Jackson Atlanta International Airport",
    "Frankfurt Airport",
    "Beijing Daxing International Airport",
    "São Paulo/Guarulhos International Airport",
    "Dubai International Airport",
    "Heathrow Airport",

    "Hartsfield-Jackson Atlanta International Airport",
    "Frankfurt Airport",
```

```r
    "Beijing Daxing International Airport",
    "São Paulo/Guarulhos International Airport",
    "Dubai International Airport",
    "Heathrow Airport",

    "Hartsfield-Jackson Atlanta International Airport",
    "Frankfurt Airport",
    "Beijing Daxing International Airport",
    "São Paulo/Guarulhos International Airport",
    "Dubai International Airport",
    "Heathrow Airport",

    "Hartsfield-Jackson Atlanta International Airport",
    "Frankfurt Airport",
    "Beijing Daxing International Airport",
    "São Paulo/Guarulhos International Airport",
    "Dubai International Airport",
    "Heathrow Airport"
  ),
  passengers = c(
    42918685, 18768601, NA,       20359547, 25836771, 22111469,
    75704760, 24812849, 25051012, 24164330, 29110609, NA,
    93699630, 48918482, NA,       34466946, 66069981, 61614508,
    104653451,59355389, NA,       41307919, 86994365, 79183364,
    108067766,61564957, 49441029, NA,       92331506, 83884572,
    106302208,63189666, 53618949, 47188085, 95200000, 84463000
  ),
  stringsAsFactors = FALSE
) |>
  as_tibble() |>
  arrange(iata, year)
# Then, tidy the data, and arrange it in a way that can be properly created in a table by pi
busiest_airports_table <- busiest_airports_tidy |>
  select(year, iata, passengers) |>
  pivot_wider(names_from = year, values_from = passengers) |>
  arrange(iata)
# Create table, using all the airports as different lines on the graph, and seeing how their
busiest_airports_table
```

```
# A tibble: 6 x 7
  iata    `2020`   `2021`   `2022`    `2023`    `2024`    `2025`
  <chr>    <dbl>    <dbl>    <dbl>     <dbl>     <dbl>     <dbl>
```

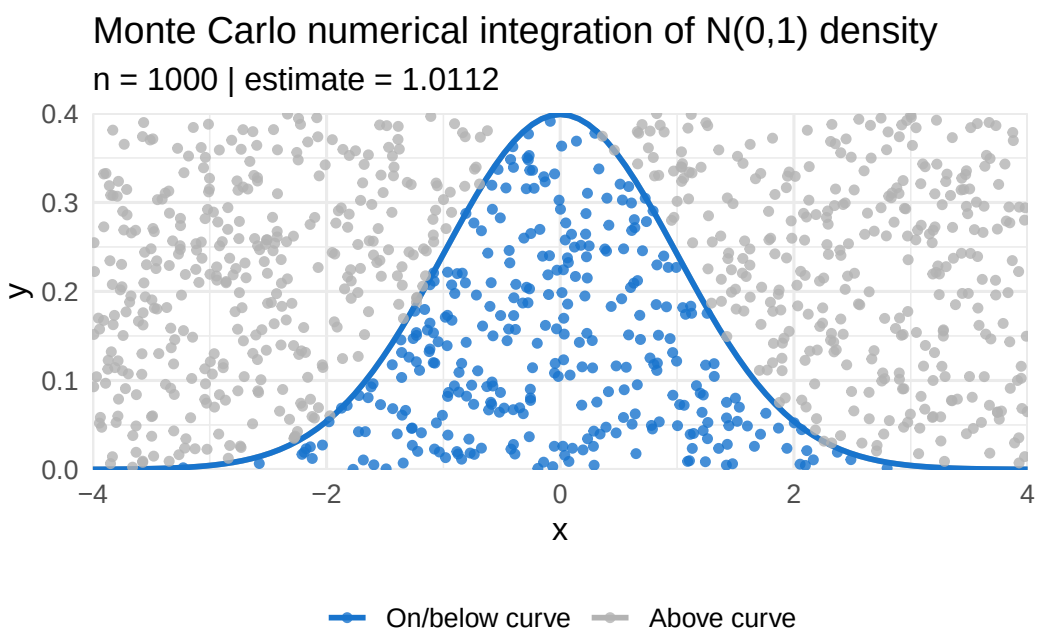
```
1 ATL   42918685 75704760 93699630 104653451 108067766 106302208
2 DXB   25836771 29110609 66069981  86994365  92331506  95200000
3 FRA   18768601 24812849 48918482  59355389  61564957  63189666
4 GRU   20359547 24164330 34466946  41307919        NA  47188085
5 LHR   22111469       NA 61614508  79183364  83884572  84463000
6 PKX         NA 25051012       NA        NA  49441029  53618949
```

```r
ggplot(busiest_airports_tidy, aes(x = year, y = passengers, color = iata, group = iata)) +
  geom_line(linewidth = 1, na.rm = TRUE) +
  geom_point(size = 2, na.rm = TRUE) +
  scale_y_continuous(labels = label_number(scale_cut = cut_short_scale())) +
  scale_x_continuous(breaks = 2020:2025) +
  labs(
    title = "Passenger Traffic (2020-2025) for Selected Busy Airports",
    x = "Year",
    y = "Total passengers",
    color = "Airport (IATA)"
  ) +
  theme_minimal(base_size = 12)
```



Code for monte carlo:

```r
library(dplyr)
library(ggplot2)
#Make a simulation, where you can specify the bounds of the function, and it randomly generat
mc_generate_points <- function(n, x_min, x_max, y_min, y_max) {
  stopifnot(length(n) == 1, n > 0)
  stopifnot(length(x_min) == 1, length(x_max) == 1, x_min < x_max)
  stopifnot(length(y_min) == 1, length(y_max) == 1, y_min < y_max)
  data.frame(
    x = stats::runif(n, min = x_min, max = x_max),
    y = stats::runif(n, min = y_min, max = y_max)
  )
}

n <- 1000
x_min <- -4; x_max <- 4
y_min <- 0;  y_max <- 0.40
# Calculate the rectangular area of the specified bounds
rect_area <- (x_max - x_min) * (y_max - y_min)
# Generate the points on the graph, and check if they're under the curve of the specified fu
res <- mc_generate_points(n, x_min, x_max, y_min, y_max) %>%
  mutate(
    f = dnorm(x, mean = 0, sd = 1),
    under_curve = y <= f
  )
# Estimate the area based on the points and if they're under the function.
p_hat <- mean(res$under_curve)
mc_estimate <- p_hat * rect_area
#Create a graph, that shows the function, and overlays all the points generated on top of it
ggplot(res, aes(x = x, y = y, color = under_curve)) +
  stat_function(
    fun = dnorm,
    args = list(mean = 0, sd = 1),
    xlim = c(x_min, x_max),
    linewidth = 1
  ) +
  geom_point(size = 1.2, alpha = 0.8) +
  scale_color_manual(
    values = c(`TRUE` = "dodgerblue3", `FALSE` = "gray70"),
    breaks = c(TRUE, FALSE),
    labels = c("On/below curve", "Above curve"),
    name = NULL
  ) +
```

```r
  coord_cartesian(xlim = c(x_min, x_max), ylim = c(y_min, y_max), expand = FALSE) +
  labs(
    title = "Monte Carlo numerical integration of N(0,1) density",
    subtitle = paste0("n = ", n, " | estimate = ", signif(mc_estimate, 6)),
    x = "x",
    y = "y"
  ) +
  theme_minimal(base_size = 12) +
  theme(legend.position = "bottom")
```

```
Warning: Multiple drawing groups in `geom_function()`
i Did you use the correct group, colour, or fill aesthetics?
```